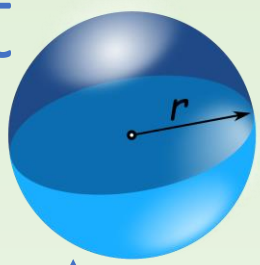
Tervezési minták II.
1.rész
Testek térfogata

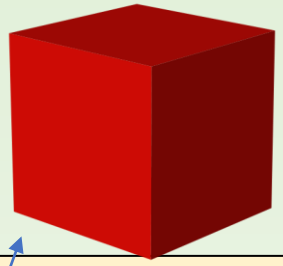
Gregorics Tibor

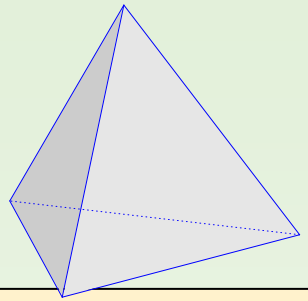
gt@inf.elte.hu

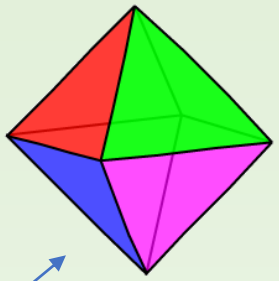
http://people.inf.elte.hu/gt/oep

1.Feladat





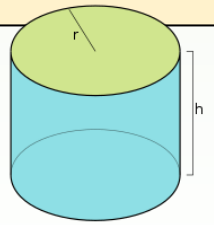


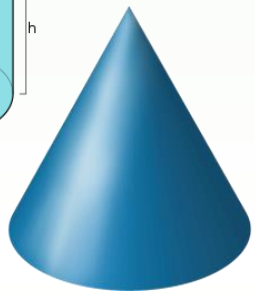


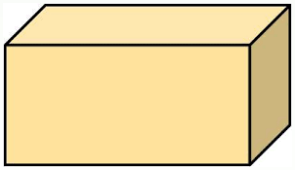
Készítsünk programot, amellyel különféle testeket hozhatunk létre, hogy kiszámolhassuk azok térfogatát, és megadhassuk, hogy az egyes testfajtákból hány példánnyal rendelkezünk már!

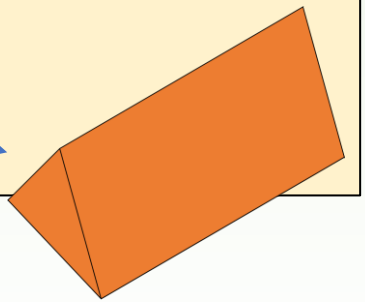
A lehetséges testfajták:

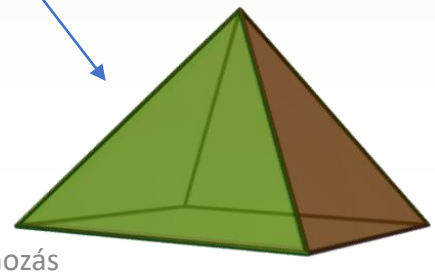
- szabályos testek: gömb, kocka, tetraéder, oktaéder;
- hasáb jellegű testek: henger, négyzet alapú hasáb és szabályos háromszög alapú hasáb;
- gúla jellegű testek: kúp, négyzetes gúla.



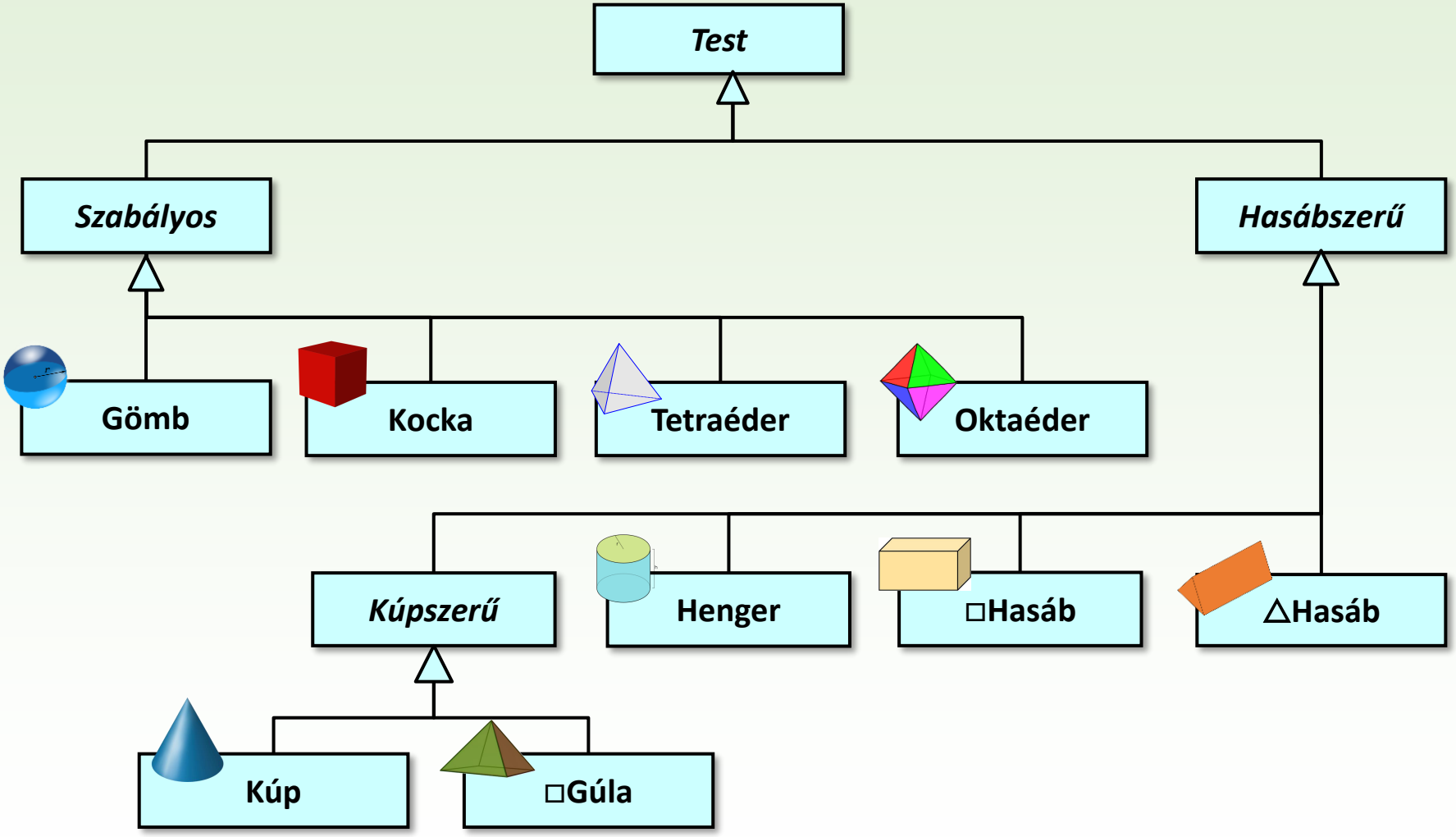




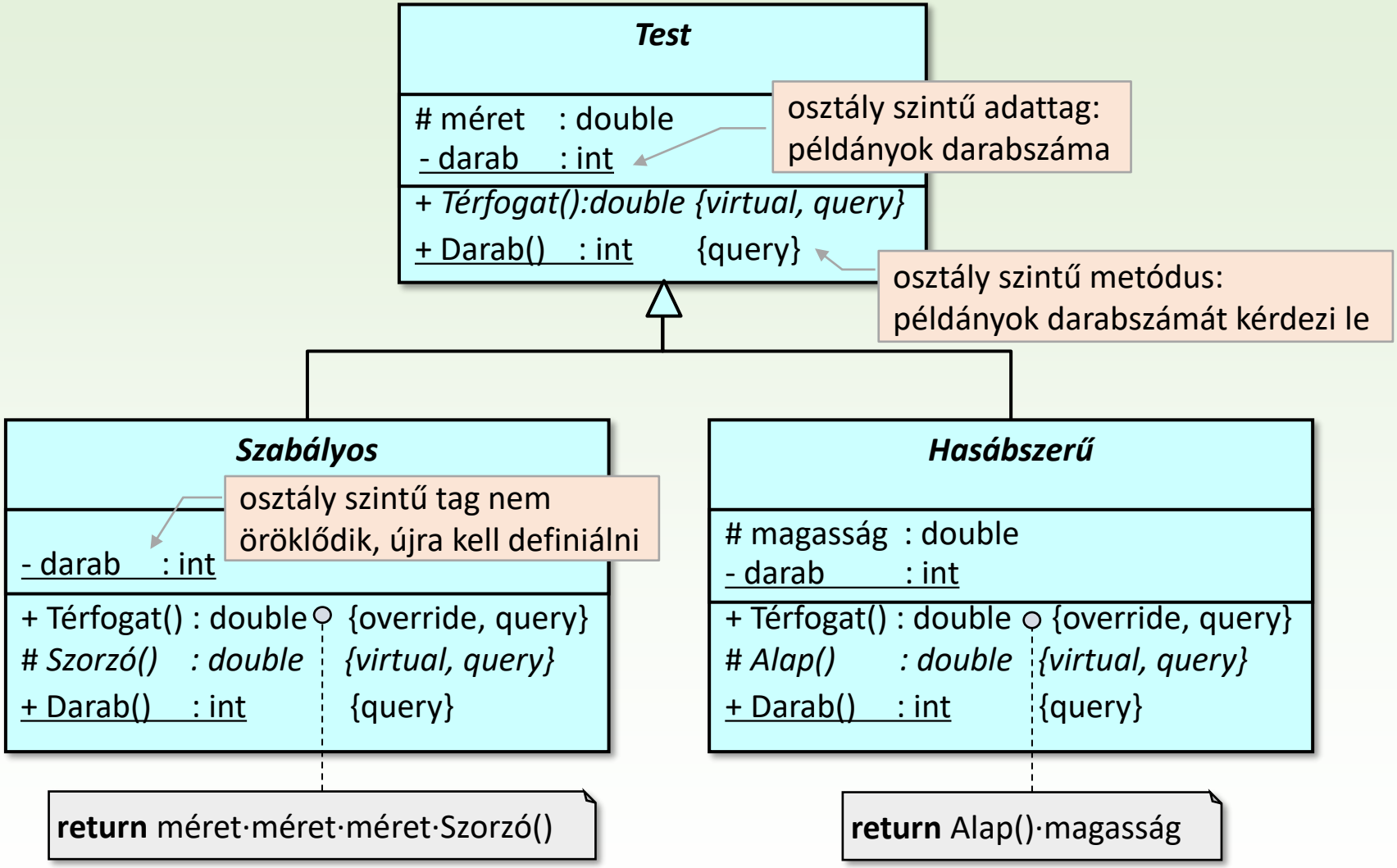




Osztálydiagram



Absztrakt testek



A testek ősosztálya

absztrakt osztály, mert konstruktora nem publikus, és van absztrakt metódusa (volume)

```
class Shape
{
        public:
                virtual ~Shape();
                virtual double volume() const = 0;
                static int piece() { return _piece; }
        protected:
                Shape(double size);
                double _size;
        private:
                static int _piece;
};
```

virtuális a destruktor

osztályszintű metódus

osztályszintű adattag

```
int Shape::_piece = 0;

Shape::Shape(double size){
        _size = size;
        ++_piece;
}
Shape::~Shape(){
        --_piece;
}
```

osztályszintű adattag kezdeti értékadása

osztályszintű adattag változtatása

Szabályos absztrakt test

```
class Regular : public Shape{
    public:
        ~Regular();
        double volume() const override;
        static int piece() { return _piece; }
    protected:
        Regular(double size);
        virtual double multiplier() const = 0;
    private:
        static int _piece;
};
```

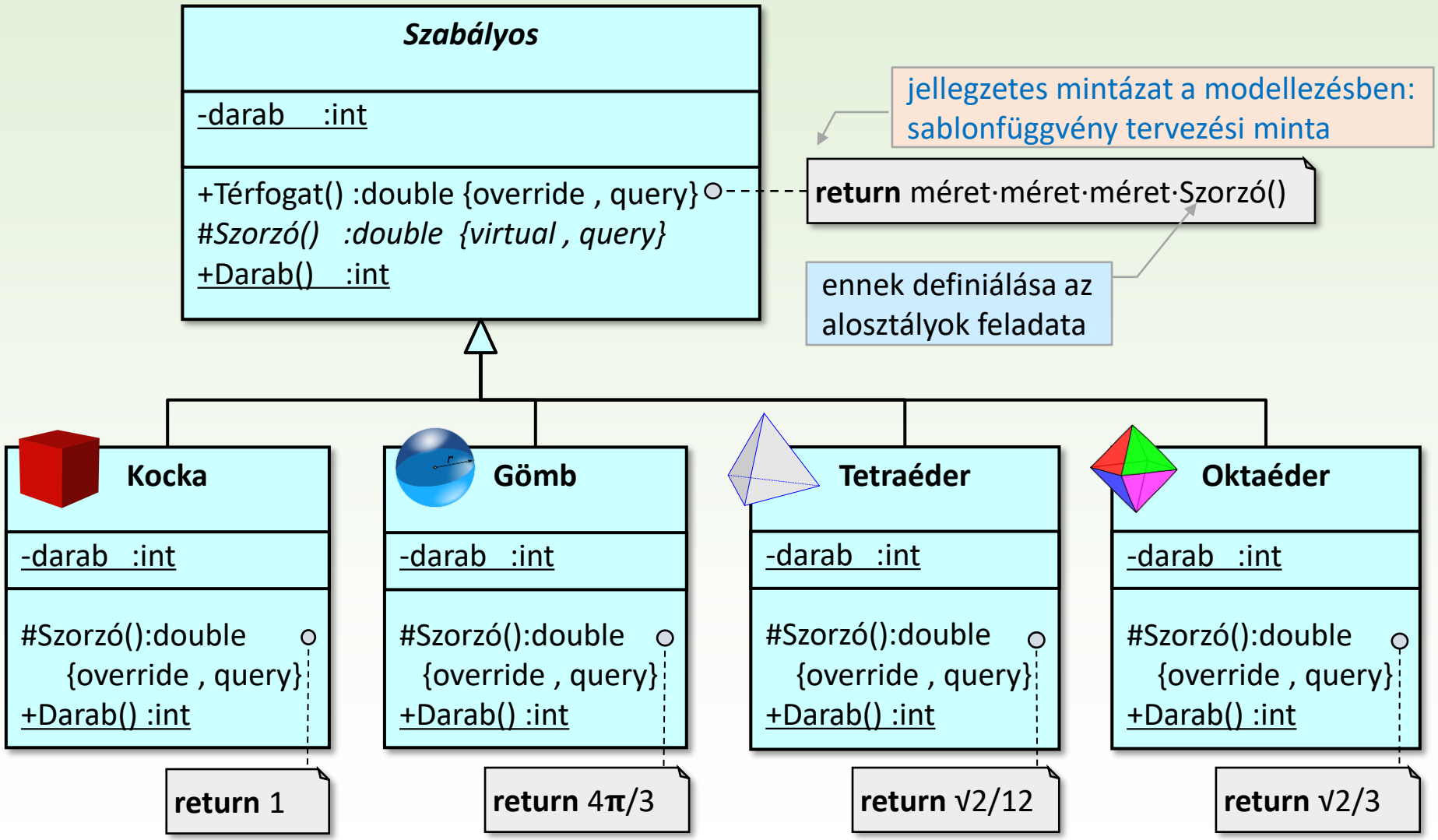
```
int Regular::_piece = 0;

Regular::Regular(double size) : Shape(size){
    ++_piece;
}
Regular::~Regular(){
    --_piece;
}
double Regular::volume() const {
    return _size * _size * _size * multiplier();
}
```

Ez hívja meg az ősosztály konstruktorát, amely inicializálja a size adattagot. Enélkül a Regular() automatikusan hívná az ősosztály üres (paraméter nélküli) konstruktorát, de olyan nincs.

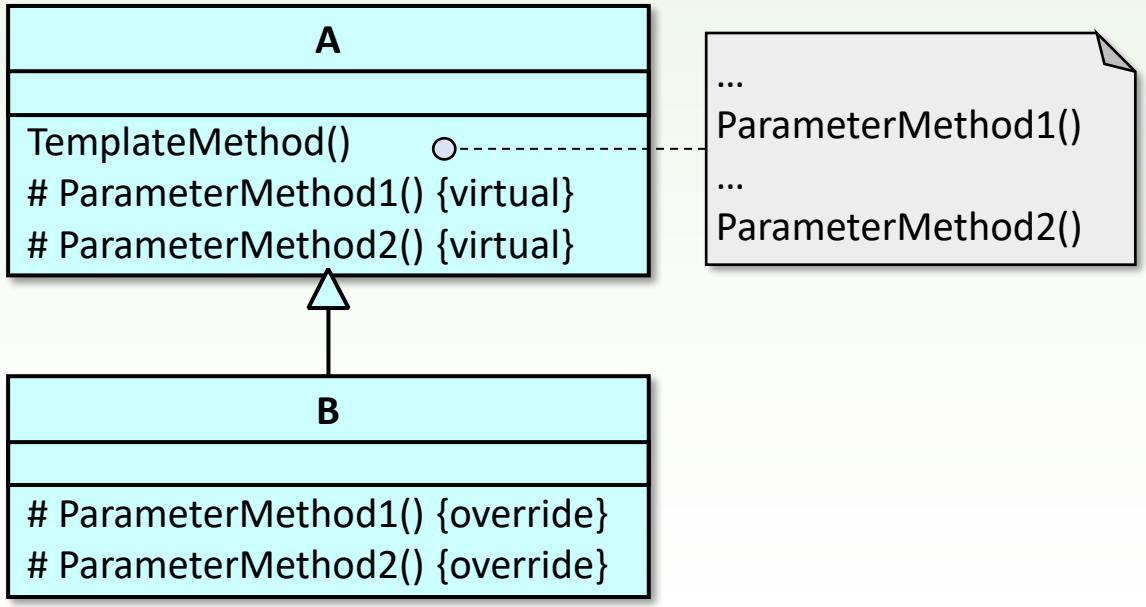
a destruktor automatikusan hívja az ősosztály destruktorát

Szabályos testek

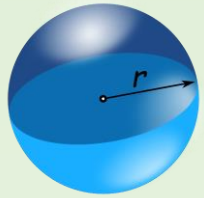


Sablonfüggvény tervezési minta (template method)

- Egy algoritmust úgy adunk meg egy osztály metódusaként, hogy annak egyes lépéseit az algoritmus szerkezetének változtatása nélkül a futási idejű polimorfizmusra támaszkodva lehessen megváltoztatni.



Gömb



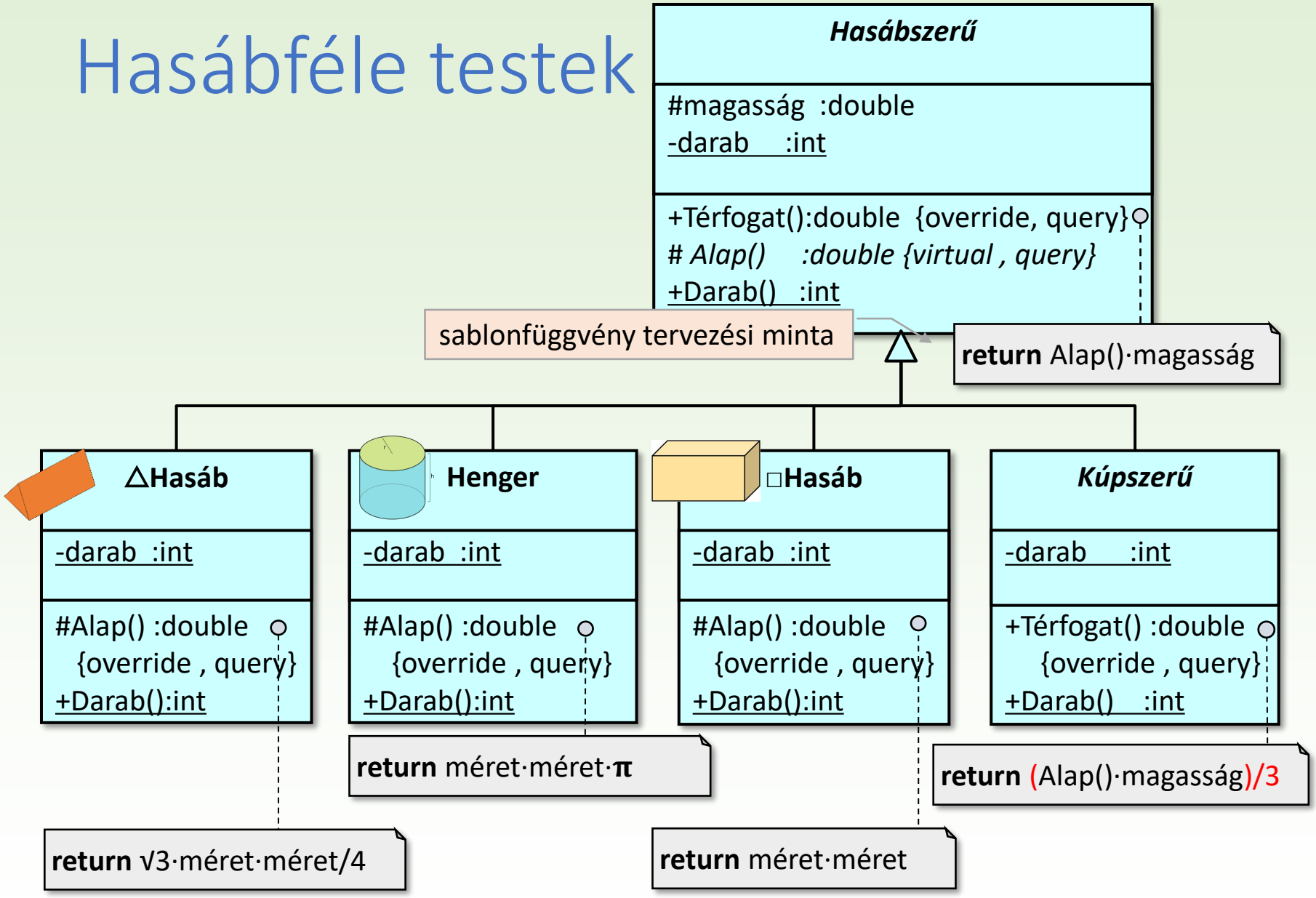

```
class Sphere : public Regular
{
	public:
		Sphere(double size);
		~Sphere();
		static int piece() { return _piece; }
	protected:
		double multiplier() const override { return _multiplier; }
	private:
		constexpr static double _multiplier = (4.0 * 3.14159) / 3.0;
		static int _piece;
};
```

konstans osztályszintű kifejezés definiálása

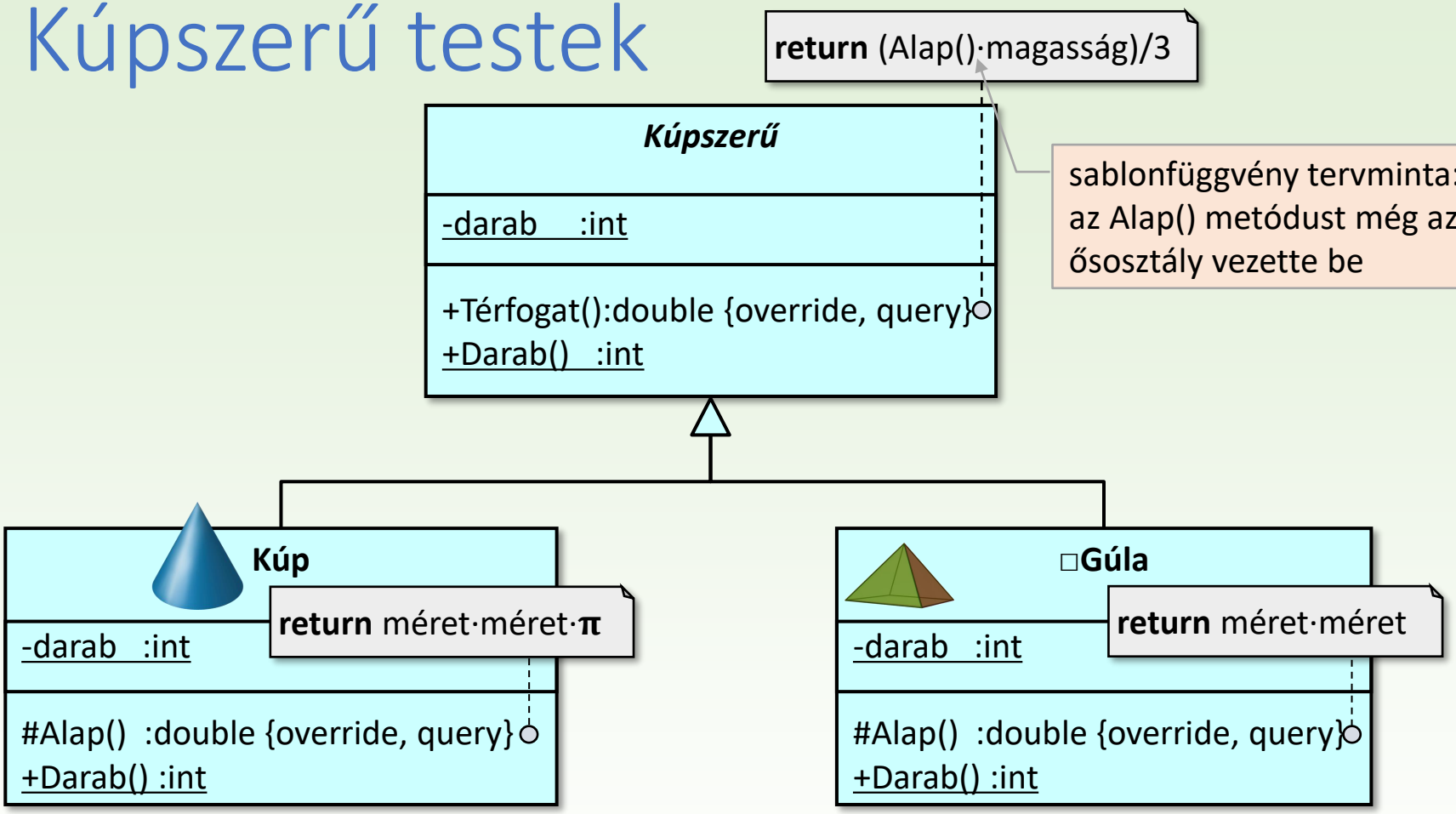
```
int Sphere::_piece = 0;

Sphere::Sphere(double size) : Regular(size){
	++_piece;
}
Sphere::~Sphere(){
	--_piece;
}
```

Hasábféle testek



Kúpszerű testek



Főprogram

```
int main()
{
	ifstream inp("shapes.txt");
	vector<Shape*> shapes;
	Shape* sh = nullptr;
	while( Shape::create(inp, sh) )
	{
		shapes.push_back(sh);
	}
	statistic();

	for ( Shape* sh : shapes )
	{
		cout << sh->volume() << endl;
	}

	for ( Shape* sh : shapes ) delete sh;
	statistic();

	return 0;
}
```

osztályszintű gyártó függvény

shapes.txt

```
Cube 5.0
Cylinder 3.0 8.0
Cylinder 1.0 10.0
Tetrahedron 4.0
SquarePyramid 3.0 10.0
Octahedron 1.0
Cube 2.0
SquarePyramid 2.0 10.0
```

vector<Shape> nem lenne jó, mert
1. a Shape absztrakt
2. a Shape-nek nincs üres konstruktora
3. a futási idejű polimorfizmus kihasználása miatt a Shape leszármazottjainak referenciáit vagy pointereit kellene tárolni

A futási idejű polimorfizmus miatt ez a shape által hivatkozott test volume() metódusa, nem a Shape ősosztályé.

A futási idejű polimorfizmus miatt ez a shape által hivatkozott test destruktora, nem a Shape ősosztályé.

Test példányosítása

shapes.txt

```
Cube 5.0
Cylinder 3.0 8.0
Cylinder 1.0 10.0
Tetrahedron 4.0
SquarePyramid 3.0 10.0
Octahedron 1.0
Cube 2.0
SquarePyramid 2.0 10.0
```

```
bool Shape::create(ifstream &inp, Shape* &sh)
{
   string type;
   double size, height;

   if( inp >> type ) {
      inp >> size;
      if ( type == "Cube" )                sh = new Cube(size);
      else if ( type == "Sphere" )         sh = new Sphere(size);
      else if ( type == "Tetrahedron" ) sh = new Tetrahedron(size);
      else if ( type == "Octahedron" )  sh = new Octahedron(size);
      else {
         inp >> height;
         if ( type == "Cylinder" )              sh = new Cylinder(size, height);
         else if ( type == "SquarePrism" )      sh = new SquarePrism(size, height);
         else if ( type == "TriangularPrism" ) sh = new TriangularPrism(size, height);
         else if ( type == "Cone" )             sh = new Cone(size, height);
         else if ( type == "SquarePyramid" )  sh = new SquarePyramid(size, height);
         else throw new UnknownShapeExeption();
      }
      return true;
   }
   return false;
}
```

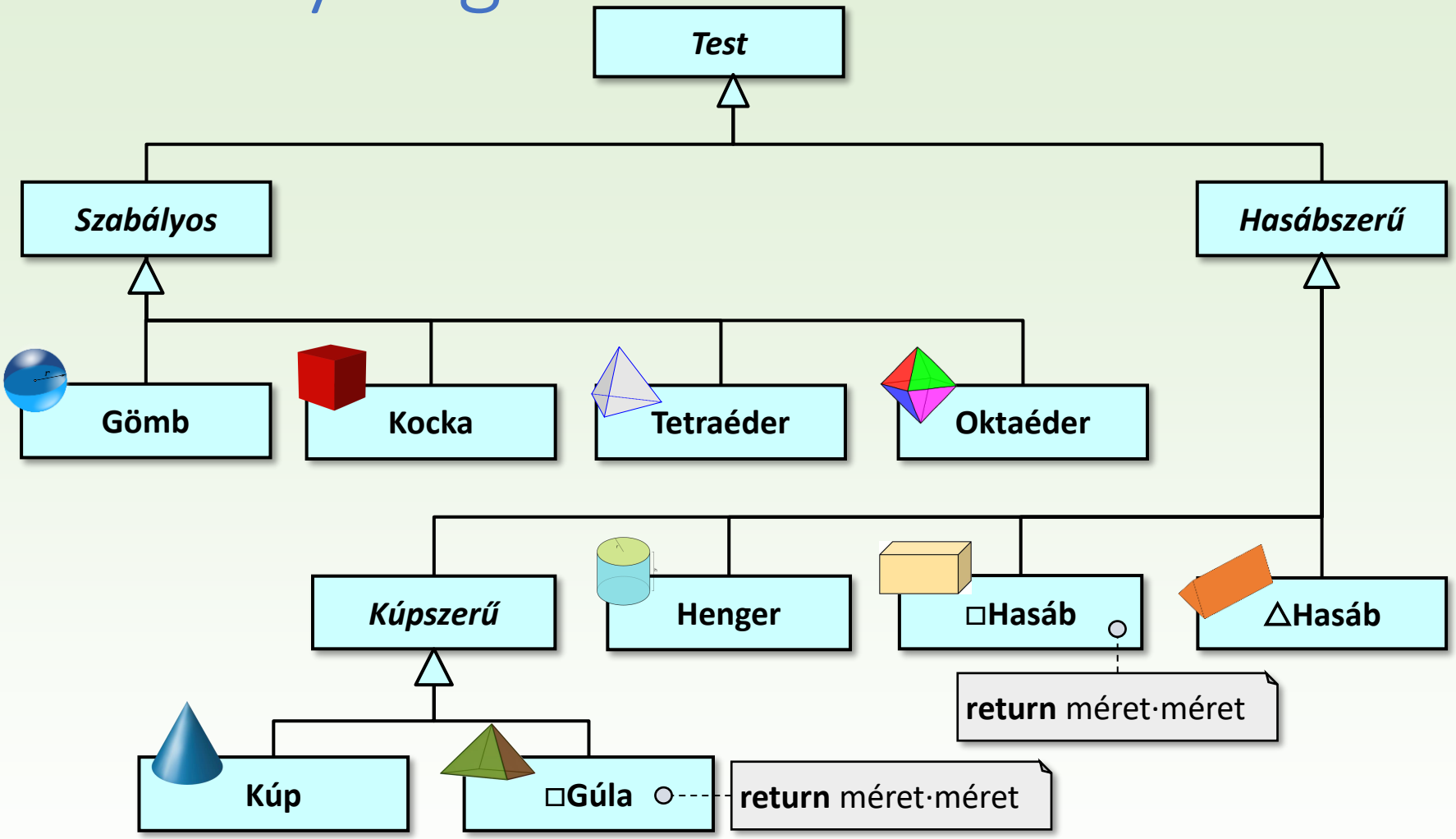
osztályszintű gyártó függvény

A származtatás miatt lehet értékül adni egy Shape* típusú változónak egy Cube* pointert

Statisztika kiírása

```
void statistic(){
      cout << Shape::piece()       << " " << Regular::piece()       << " "
           << Prismatic::piece()   << " " << Conical::piece()       << " "
           << Sphere::piece()      << " " << Cube::piece()          << " "
           << Tetrahedron::piece() << " " << Octahedron::piece()    << " "
           << Cylinder::piece()    << " " << SquarePrism::piece()   << " "
           << TriangularPrism::piece()    << " "
           << Cone::piece()        << " " << SquarePyramid::piece()<< endl;
}
```

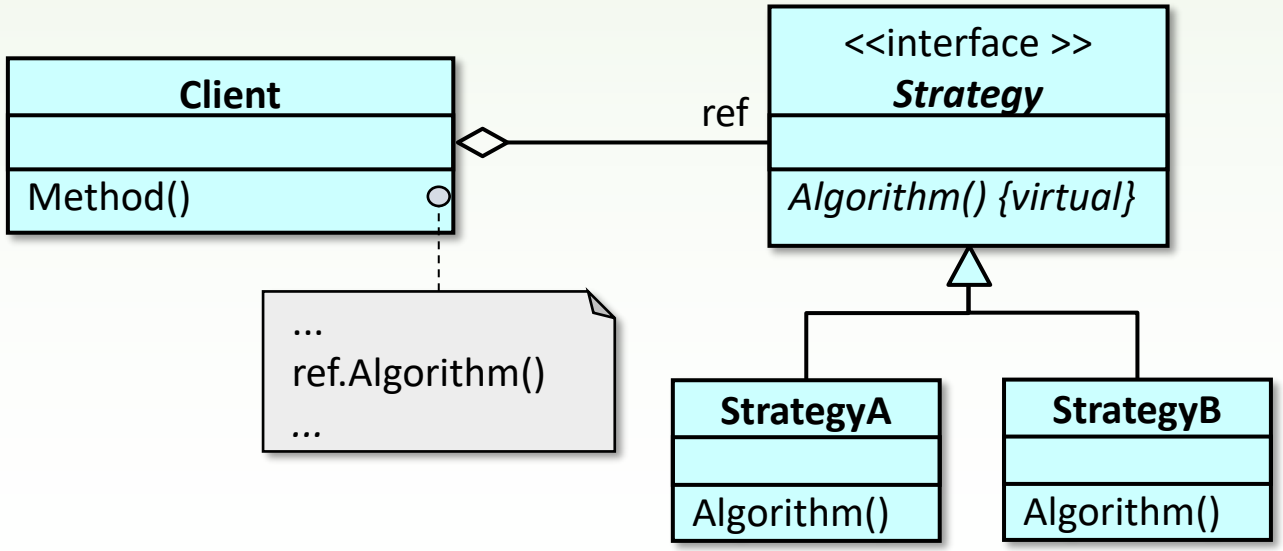
Osztálydiagram



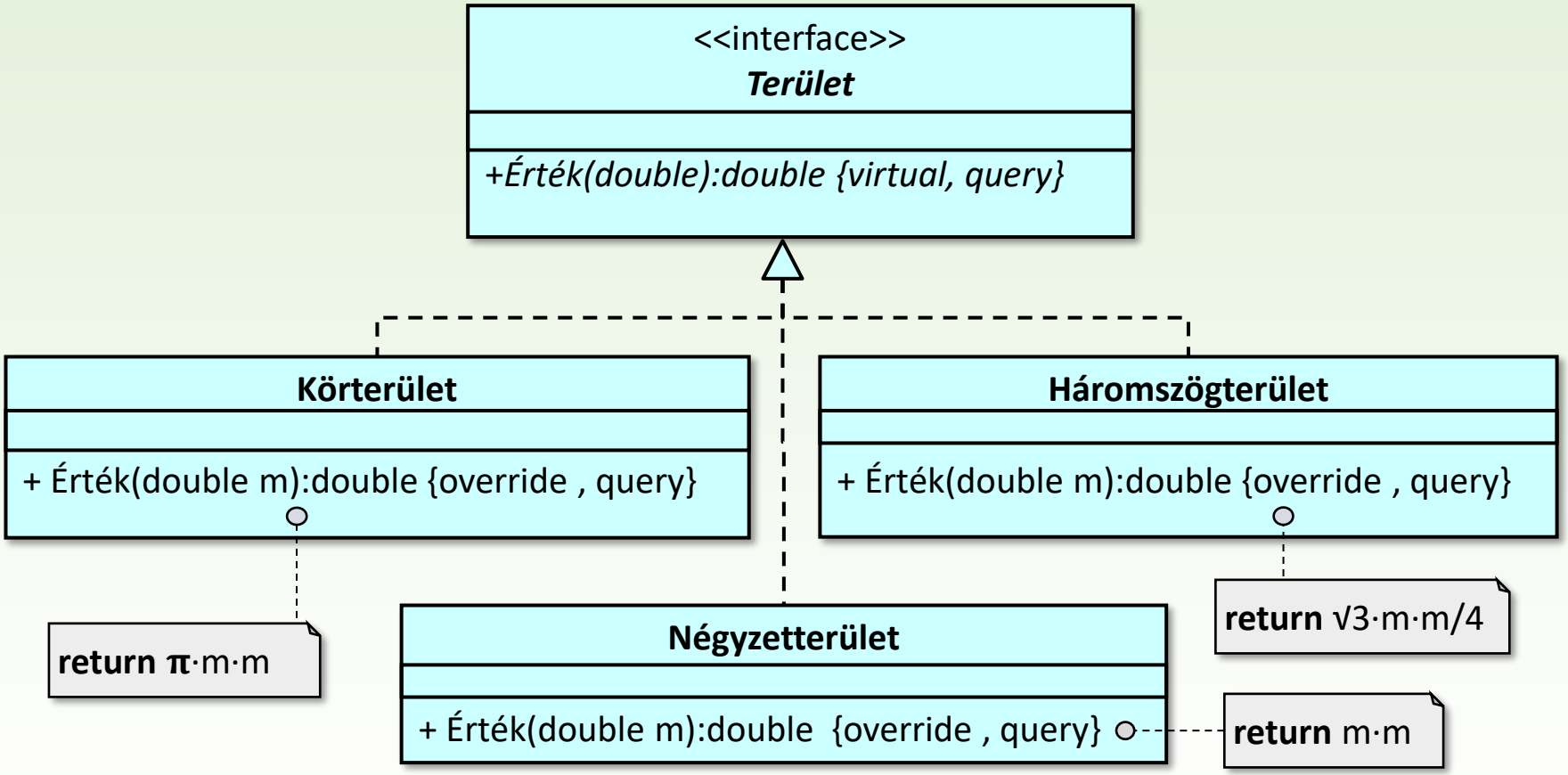
Kritika a modellről: redundancia (kódismétlődés) jelent meg a modellben. Ugyanazon alapterület kiszámolása több osztályban is szerepel: kör a kúp és a henger osztályaiban, négyzet a négyzetalapú hasáb és gúla osztályaiban.

Stratégia (strategy) tervezési minta

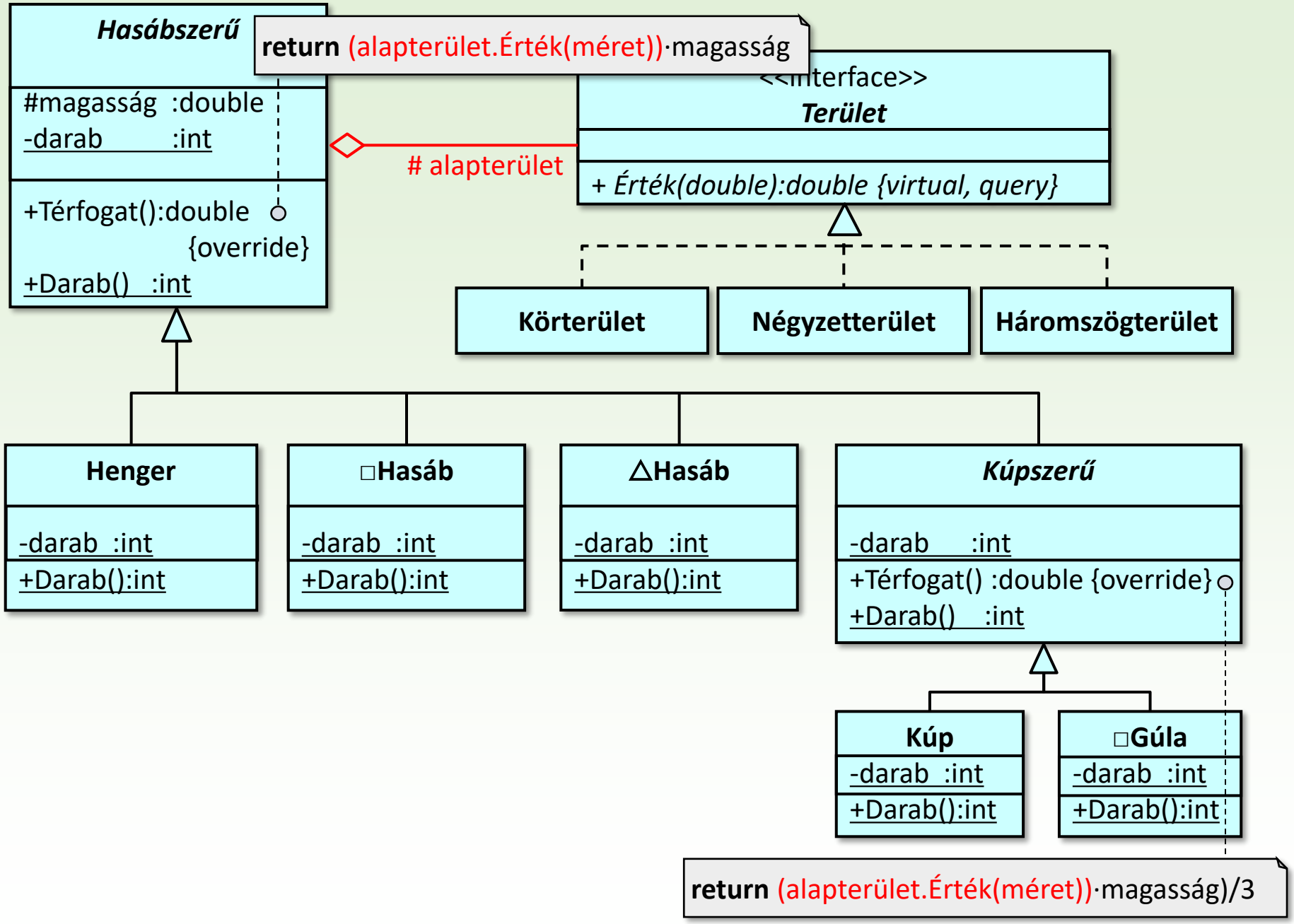
- Tevékenységek (algoritmusok) családját definiáljuk egy közös interfészt megvalósító osztályok metódusaként azért, hogy egy futási időben dőlhessen el az, hogy melyiket használjuk fel egy másik (kliens) osztály egyik metódusában. Egy-egy tevékenységet egy objektum hordoz a saját metódusaként, és ezt az objektumot aggregáljuk be a kliens osztály egy-egy objektumába (vagy a kliens osztály metódusa is megkaphatja paraméterként).



Alapterületet számoló objektumok



Itt, és csak itt kell az alapterületeket definiálni – megszűnik a redundancia.


Hasábszerű
#magasság :double
-darab :int
+Térfogat():double {override}
+Darab() :int
return (alapterület.Érték(méret))·magasság
alapterület
<<interface>>
Terület
+ Érték(double):double {virtual, query}
Körterület
Négyzetterület
Háromszögterület
Henger
-darab :int
+Darab():int
□Hasáb
-darab :int
+Darab():int
△Hasáb
-darab :int
+Darab():int
Kúpszerű
-darab :int
+Térfogat() :double {override}
+Darab() :int
Kúp
-darab :int
+Darab():int
□Gúla
-darab :int
+Darab():int
return (alapterület.Érték(méret))·magasság)/3

Alapterület objektum használata

```
class Prismatic : public Shape
{
public:
    ~Prismatic ();
    double volume() const;
    static int piece() { return _piece;    }
protected:
    Prismatic (double size, double height);
    Area * _baseArea;
    double _height;
private:
    static int _piece;
};

double Prismatic::volume() const
{
    return _baseArea->value(_size) * _height;
}
```

az alapterület objektum hivatkozásának tárolására alkalmas adattag

az alapterület objektum területszámító metódusa

```
Cylinder::Cylinder(…) : Prismatic(…) {
        ++_piece; _baseArea = new CircleArea();
}
Cylinder::~Cylinder(){
        --_piece; delete _baseArea;
}
```

az alapterület adattagot a konstruktor állítja be, destruktor törli

```
Cone::Cone(...) : Conical(…){
        ++_piece; _baseArea = new CircleArea();
}
Cone::~Cone(){
        --_piece; delete _baseArea;
}
```

```
SquarePrism::SquarePrism(…) : Prismatic(…){
        ++_piece; _baseArea = new SquareArea();
}
SquarePrism::~SquarePrism(){
        --_piece; delete _baseArea;
}
```

```
SquarePyramid::SquarePyramid(…) : Conical(…){
        ++_piece; _baseArea = new SquareArea();
}
SquarePyramid::~SquarePyramid(){
        --_piece; delete _baseArea;
}
```

```
TriangularPrism::TriangularPrism(…) : Prismatic(…){
        ++_piece; _baseArea = new TriangularArea();
}
TriangularPrism::~TriangularPrism(){
        --_piece; delete _baseArea;
}
```

Megoldás

A Terület típusú objektumok legyenek egykék.

Kritika a hatékonyságról:

A kód-redundancia megszűnt, de memória-pazarlást okoz: például 5 henger és 3 kúp példányosítása összesen 8 darab körterület objektumot hoz létre, pedig egy is elég lenne.

Tervezési minták II.
2.rész
Lények túlélési versenye

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

2.Feladat

Készítsünk programot, amellyel különféle lények túlélési versenyét modellezhetjük.

A lények három faj (zöldikék, buckabogarak, tocsogók) valamelyikéhez tartoznak. Van nevük, ismert a fajuk, és az aktuális életerejük (egész szám). A versenyen induló lények sorban egymás után egy olyan pályán haladnak végig, ahol három féle (homokos, füves, mocsaras) terep váltakozik. Amikor egy lény keresztül halad egy terepen, akkor a fajától (és az adott tereptől is) függően átalakítja a terepet, miközben változik az életereje. Ha az életereje elfogy, a lény elpusztul. Adjuk meg a pálya végéig eljutó, azaz életben maradt lények neveit!

- ***Zöldike**: füvön az életereje eggyel nő, homokon kettővel csökken, mocsárban eggyel csökken; a mocsaras terepet fűvé alakítja, a másik két terep fajtát nem változtatja meg.*



- ***Buckabogár**: füvön az ereje kettővel csökken, homokon hárommal nő, mocsárban néggyel csökken; a füvet homokká, a mocsarat fűvé alakítja, de a homokot nem változtatja meg.*



- ***Tocsogó**: füvön az életereje kettővel, homokon öttel csökken, mocsárban hattal nő; a füvet mocsárrá alakítja, a másik két fajta terepet nem változtatja meg.*



Hogyan alakít át egy lény egy terepet?

átalakít : Lény×Terep ⟶ Lény×Terep



zöldikék	életerő változás	terepváltozás
homok	-2	-
fű	+1	-
mocsár	-1	fű



buckabogarak	életerő változás	terepváltozás
homok	+3	-
fű	-2	homok
mocsár	-4	fű



tocsogók	életerő változás	terepváltozás
homok	-5	-
fű	-2	mocsár
mocsár	+6	-

Megoldási terv

$pálya_{i-1}$ ~ a pálya az *i*-dik lény áthaladása előtt
$pálya_i$ ~ a pálya az *i*-dik lény áthaladása után

A pályán áthaladó lények sorban egymás után megváltoznak miközben a pálya is átalakul, majd kiválogatjuk a túlélő lények neveit.

A = (lények: Lény^n, pálya: Terep^m, túlélők: String*)

Ef = (lények = $lények_0 \wedge$ pálya = $pálya_0$)

Uf = ($\forall i \in [1..n]$: (lények[i], $pálya_i$) = áthalad($lények_0$[i], $pálya_{i-1}$) $\wedge$ pálya = $pálya_n$

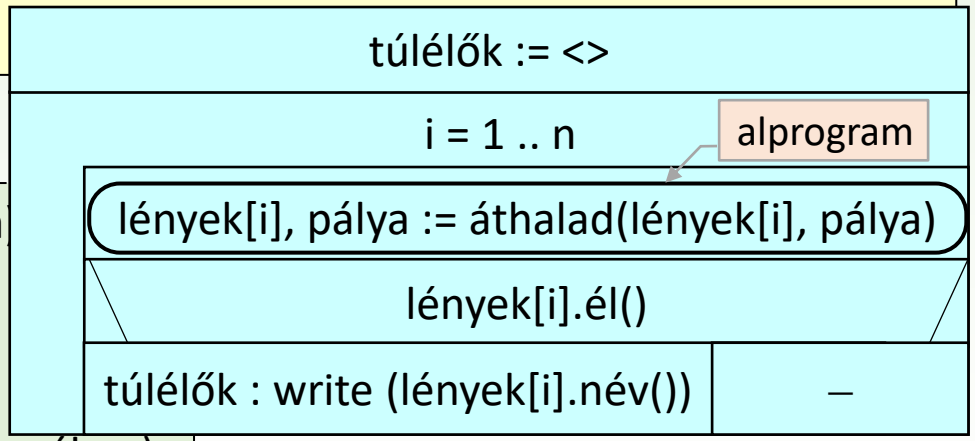
$\wedge$ túlélők = $\bigoplus_{i=1..n}$ <lények[i].név()>)

lények[i].él()

Szimuláció:: minden lény áthalad az előző lények által már megváltoztatott pályán: a lény is, a pálya is megváltozik

Kiválogatás: túlélő lények kiválogatása

A megváltozott lények tömbjét összefűzés helyett az eredeti tömb megfelelő indexű elemeinek felülírásával alakítjuk ki



t:enor(E) ~ i = 1 .. n (lények felsorolása)

szimuláció - dupla összegzés

összefűzés és sorozatos lecserélés :

f(e) ~ áthalad(lények[i], pálya)

H,+,0 ~ Lény^n×Terep^m, ($\bigoplus$, $\ominus$), (<>, $pálya_0$)

kiválogatás - összegzés

f(e) ~ <lények[i].név()> ha lények[i].él()

H,+,0 ~ String*, $\bigoplus$, <>

régi $\ominus$ új ::= új

A két tevékenység közös ciklusba vonható össze

Egy lény áthaladása

Egy lény (lények[i]) az aktuális pálya terepein halad addig, amíg él: minden lépése megváltoztathatja a terepet, miközben a lény maga is átalakul.

A = (pálya: Terep^m, lény: Lény)

Ef = (pálya = pálya' $\wedge$ lény = lény$_0$)

Uf = ($\forall j \in [1..m]$: (lény$_j$, pálya[j]) = átalakít(lény$_{j-1}$, pálya'[j]) $\wedge$ lény = lény$_m$)

lény$_{j-1}$ ~ a lény a *j*-dik terep átalakítása előtt
lény$_j$ ~ a lény a *j*-dik terep átalakítása után

Áthaladás: a pálya terepeit egymás után alakítja át a lény, amelynek állapota sorozatosan megújul.

áthaladás - feltételig tartó dupla összegzés (sorozatos lecserélés és összefűzés):

t:enor(E) ~ j = 1 .. m amíg lény.él()

H,+,0 ~ Lény×Terep^m, ($\ominus$, $\oplus$), (lény$_0$, <>)

f(e) ~ átalakít(lény, pálya[j])

A megváltozott lények tömbjét összefűzés helyett az eredeti tömb megfelelő indexű elemeinek felülírásával alakítjuk ki

alprogram

lény, pálya := áthalad(lény, pálya)

j := 1

lény.él() $\wedge$ j ≤ m

lény.átalakít(pálya[j])

j := j+1

lény, pálya[j] := átalakít(lény, pálya[j])

Főprogram

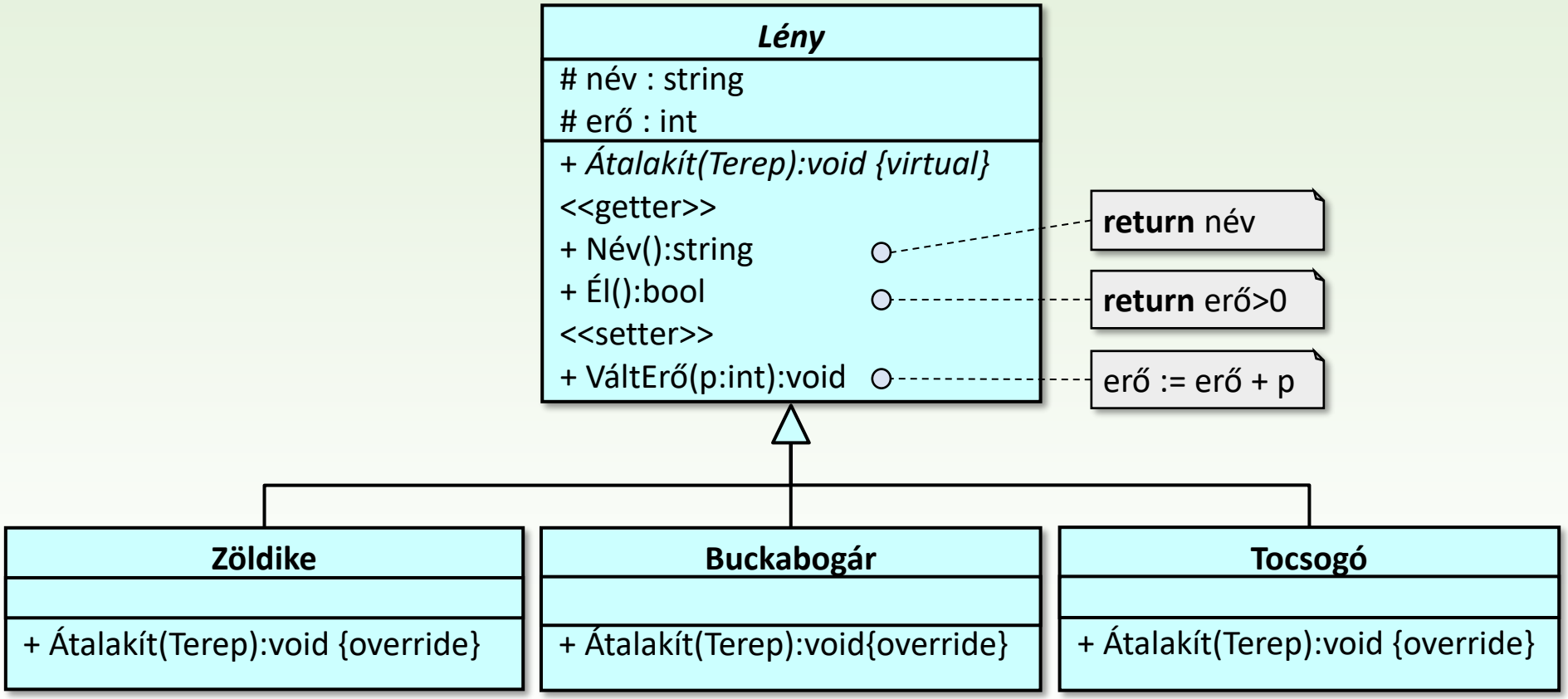
```
// populating
…
// competition
for ( int i=0; i < n; ++i ){
        for ( int j=0; creatures[i]->alive() && j < m; ++j ){
                creatures[i]->transmute(courts[j]);
        }
        if (creatures[i]->alive() ) cout << creatures[i]->name() << endl;
}

// destroying
…
```

main.cpp

lény.átalakít(pálya[j])
Itt jól jönne a futási idejű polimorfizmus, azaz a transmute() működése attól függjön, hogy a creatures[i] milyen fajtájú lény, azaz mi a típusa.

Lények származtatása



Lények átalakít() metódusai

```
void Greenfinch::transmute(int &court) {
    if ( alive() ){
        switch( court ){
            case 0: _power-=2; break;
            case 1: _power+=1; break;
            case 2: _power-=1; court = 1; break;
        }
    }
}
```

```
void DuneBeetle::transmute(int &court) {
    if ( alive() ){
        switch( court ){
            case 0: _power+=3; break;
            case 1: _power-=2; court = 0; break;
            case 2: _power-=4; court = 1; break;
        }
    }
}
```

```
void Squelchy::transmute(int &court) {
    if ( alive() ){
        switch( court ){
            case 0: _power-=5; break;
            case 1: _power-=2; court = 2; break;
            case 2: _power+=6; break;
        }
    }
}
```

creature.cpp

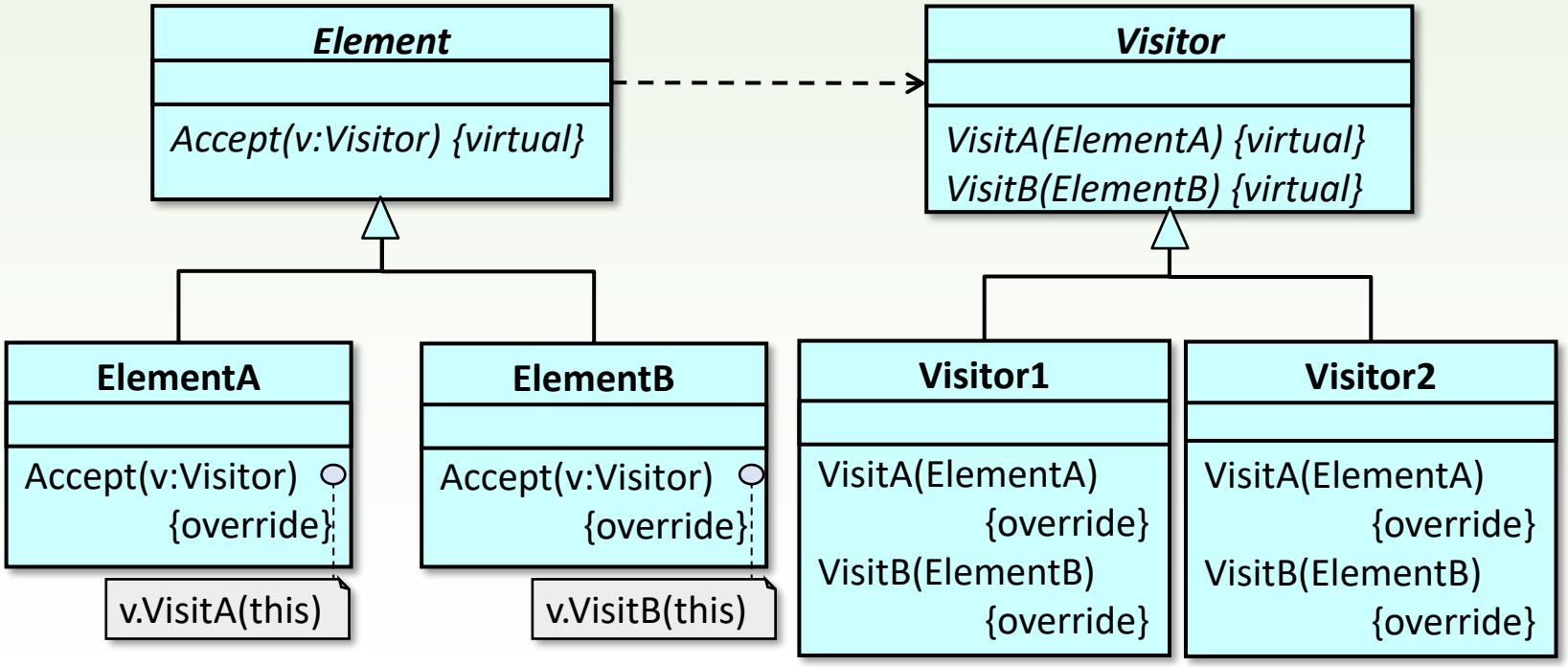
Kritika a kódról:

- sérül az Open/Close elv:
  újabb terepfajta bevezetése esetén az összes elágazást ki kell bővíteni, azaz meglévő kódon kell változtatni
- nem eléggé beszédes a kód:
  a terepeket azonosító egész számok nem „beszédes” jelölések

bevezethetnénk az alábbi típust:
enum Ground { sand, grass, marsh },
hogy kiváltsuk a 0, 1, 2 használatát,
de nem ezt csináljuk.

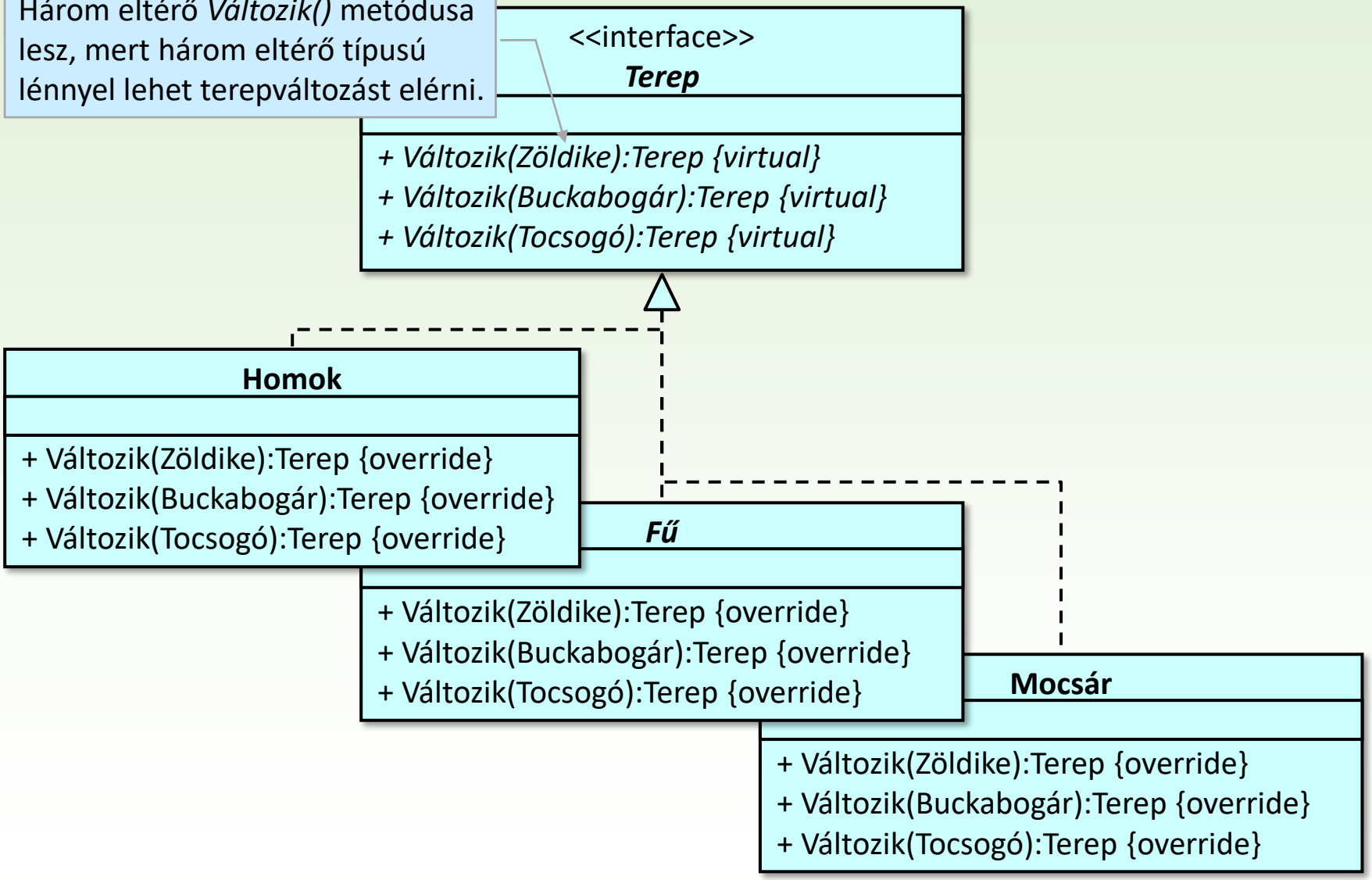
Látogató (visitor) tervezési minta

- Amikor egy metódus működése a saját osztályán kívül attól is függ, hogy a neki adott paraméter-objektum egy objektum-készlet melyik osztályú objektuma, viszont nem akarjuk, hogy ez a függőség (például elágazás formájában) megjelenjen kódban.



Terepek származtatása

Három eltérő *Változik()* metódusa lesz, mert három eltérő típusú lénnyel lehet terepváltozást elérni.



Terep változása egy lény által

```
Ground* Sand::change(Greenfinch *p){
   p->changePower(-2); return this;
}
Ground* Sand::change(DuneBeetle *p){
   p->changePower(3);  return this;
}
Ground* Sand::change(Squelchy *p){
   p->changePower(-5); return this;
}
```

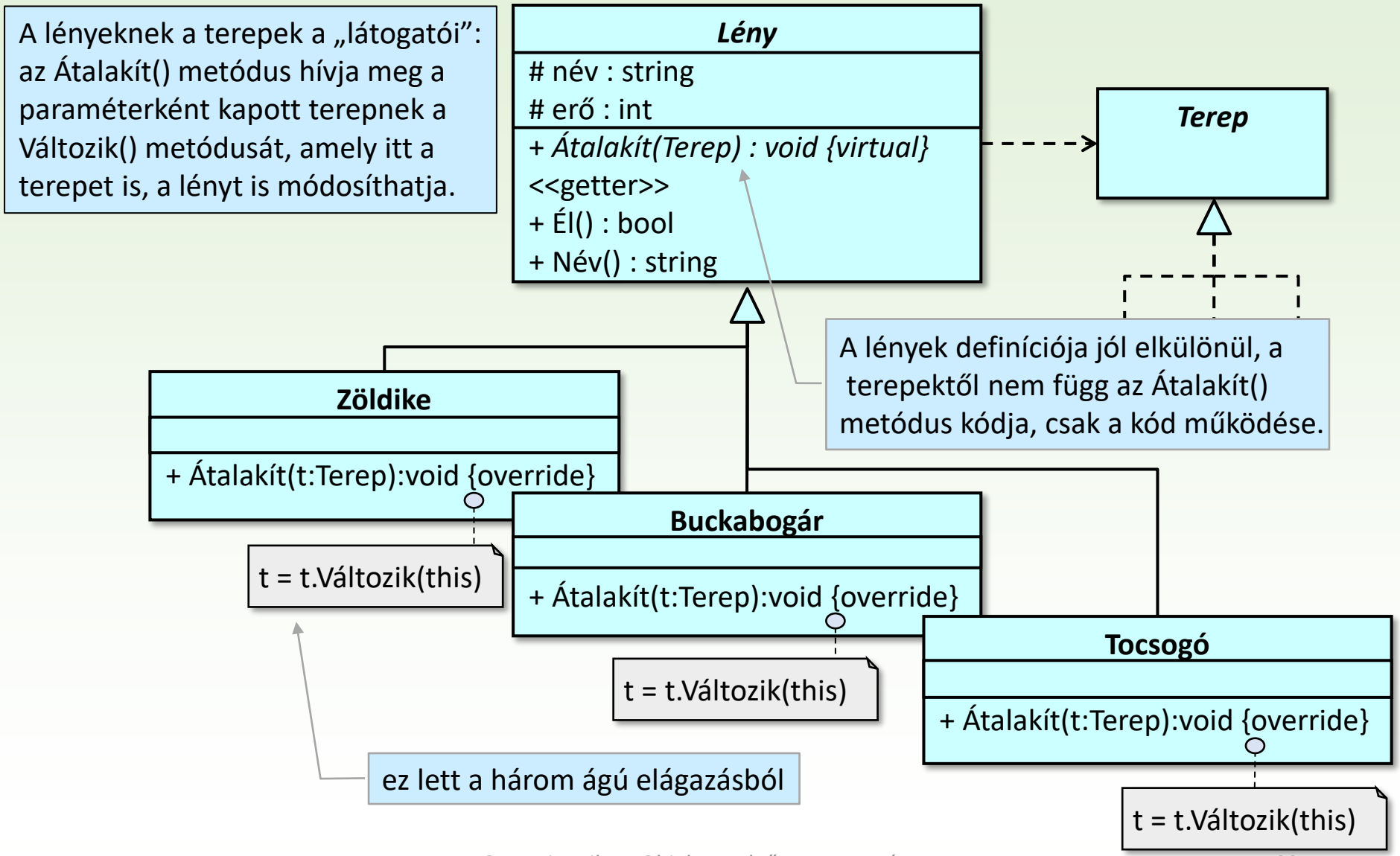
```
Ground* Grass::change(Greenfinch *p){
   p->changePower(1);  return this;
}
Ground* Grass::change(DuneBeetle *p){
   p->changePower(-2); return new Sand;
}
Ground* Grass::change(Squelchy *p){
   p->changePower(-2); return new Marsh;
}
```

```
Ground* Marsh::change(Greenfinch *p){
   p->changePower(-1); return new Grass;
}
Ground* Marsh::change(DuneBeetle *p){
   p->changePower(-4); return new Grass;
}
Ground* Marsh::change(Squelchy *p){
   p->changePower(6);  return this;
}
```

ground.cpp

Itt összesen 9 darab, lénytől és tereptől függő change() metódus van.
Újabb terepfajta esetén 1 új osztályt kell 3 új change() metódussal definiálni.
Újabb lényfajta esetén minden osztályba 1-1 újabb change() metódust kell betenni.
De meglevő kódon nem kell változtatni.

Lények osztálydiagramja újra



Lények látogatói a terepek

```
class Creature{
protected:
        int _power;
        std::string _name;
        Creature (const std::string &str, int e = 0)
        :_name(str), _power(e) {}
public:
        std::string name() const { return _name; }
        bool alive() const { return _power > 0; }
        void changePower(int e) { _power += e; }
        virtual void transmute(Ground* &court) = 0;
        virtual ~Creature () {}
};
class Greenfinch : public Creature {
public:
        Greenfinch(const std::string &str, int e = 0) : Creature(str, e){}
        void transmute(Ground* &court) override {court = court->change(this);}
};
class DuneBeetle : public Creature {
public:
        DuneBeetle(const std::string &str, int e = 0) : Creature(str, e){}
        void transmute(Ground* &court) override {court = court->change(this);}
};
class Squelchy : public Creature {
public:
        Squelchy(const std::string &str, int e = 0) : Creature(str, e){}
        void transmute(Ground* &court) override {court = court->change(this);}
};
```

Kritika a hatékonyságról:
elég lenne minden fajta terephez egy-egy objektum, amelyre több helyről is hivatkozhatunk, de a change() mindig új objektumot példányosít, valahányszor egy terep változik, miközben a régi terep-objektum pedig tovább foglalja a memóriát (memóriaszivárgás)

legyenek a terepek egykék

creature.h

Feladat felpopulálása

```
// populating
    ifstream f("input.txt");
    if (f.fail()) { cout << "Wrong file name!\n"; return 1;}
    int n; f >> n;
    vector<Creature*> creatures(n);
    for ( int i=0; i<n; ++i ){
        char ch; string name; int p;
        f >> ch >> name >> p;
        switch(ch){
            case 'G' : creatures[i] = new Greenfinch(name, p);  break;
            case 'D' : creatures[i] = new DuneBeetle(name, p); break;
            case 'S' : creatures[i] = new Squelchy(name, p);     break;
        }
    }
    int m; f >> m;
    vector<Ground*> courts(m);
    for ( int j=0; j<m; ++j ) {
        int k; f >> k;
        switch(k){
            case 0 : courts[j] = Sand::instance();  break;
            case 1 : courts[j] = Grass::instance(); break;
            case 2 : courts[j] = Marsh::instance(); break;
        }
    }
```

main.cpp

input.txt

```
4
S plash 20
G greenish 10
D bug 15
S sponge 20
10
0 2 1 0 2 0 1 0 1 2
```

```
// destroying creatures
    for (int i=0; i<n; ++i) delete creatures[i];
// destroying creatures
    Sand::destroy();
    Grass::destroy();
    Marsh::destroy();
```

Csomagok

```
#pragma once
#include "ground.h"

class Creature { … };
class Greenfinch : public Creature { …
      void transmute(Ground* &court) override
            {court = court->change(this);}
};
class DuneBeetle : public Creature { … };
class Squelchy   : public Creature { … };
```

creature.h

```
#pragma once
class Greenfinch;
class DuneBeetle;
class Squelchy;

class Ground{
public:
      virtual Ground* change(Greenfinch *g) = 0;
      virtual Ground* change(DuneBeetle *d) = 0;
      virtual Ground* change(Squelchy     *s) = 0;
};
class Sand  : public Ground { … }
class Grass : public Ground { … }
class Marsh : public Ground { … }
```

A #include "creature.h" körkörös include hivatkozást okozna. Az állatfaj osztályok deklarálásával elég azt jelezni, hogy vannak (lesznek) ilyen osztályok.

ground.h

```
#include "ground.h"
#include "creature.h"
      …
```

ground.cpp